

BUSINESS INTELLIGENCE — PROJEKTDOKUMENTATION

BI Entscheidungsagent Online Retail II

*Interaktives Dashboard mit regelbasiertem KI-Entscheidungsagenten
für einen UK-basierten Online-Händler*

Student	Sheyla Sampietro
Kurs	Business Intelligence
Dozent	Manuel Renold
Abgabedatum	April 2026
GitHub	github.com/SheylaSam/entscheidungsagent-bi
Dataset	Online Retail II (UCI ML Repository)

1. Inhaltsverzeichnis

- 1 Projektübersicht und Zielsetzung
- 2 Dataset — Online Retail II
- 3 Technologie-Entscheide
- 4 Systemarchitektur
- 5 Datenverarbeitung und Bereinigung
- 6 Analyse-Module
- 7 KI-Entscheidungsagent
- 8 Streamlit Dashboard
- 9 Testing und Qualitätssicherung
- 10 Fazit und Ausblick

2. Projektübersicht und Zielsetzung

Dieses Projekt implementiert ein vollständiges Business-Intelligence-System für einen UK-basierten Online-Händler (Geschenkartikel). Das System besteht aus drei Kernkomponenten: einer Datenpipeline, die Rohdaten aus Excel in eine relationale Datenbank überführt; einer Analyse-Schicht, die Umsatzprognosen, Kundensegmentierung und Produktperformance berechnet; sowie einem regelbasierten KI-Entscheidungsagenten, der aus diesen Analysen konkrete Managemententscheide ableitet.

Das Projektziel lässt sich in einem Satz zusammenfassen: *Aus 1,07 Millionen Transaktionen automatisch priorisierte Handlungsempfehlungen für das Management generieren.*

Anforderungen

Anforderung	Umsetzung
Datenbankanbindung	SQLite-Datenbank, automatisch aus Excel befüllt beim ersten Start
Interaktives Dashboard	Streamlit-App mit 5 Tabs, Plotly-Diagramme
KI-Entscheidungsagent	Regelbasierter Agent kombiniert Forecast + RFM + Produktdaten
Dokumentation	Dieser Bericht + README mit Tech-Entscheiden + GitHub Repo
Reproduzierbarkeit	<code>pip install -r requirements.txt && streamlit run app.py</code> genügt

3. Dataset — Online Retail II

Das verwendete Dataset stammt vom UCI Machine Learning Repository und beschreibt alle Transaktionen eines UK-basierten Online-Händlers für Geschenkartikel im Zeitraum Dezember 2009 bis Dezember 2011.

Eigenschaft	Wert
Quelle	UCI ML Repository — Online Retail II
Zeitraum	01.12.2009 – 09.12.2011
Gesamtzeilen (roh)	~1'067'000 Transaktionszeilen
Sheets	Year 2009–2010 (~525k), Year 2010–2011 (~542k)
Spalten	8 (Invoice, StockCode, Description, Quantity, InvoiceDate, Price, Customer ID, Country)
Primäres Segment	B2B-Grosshändler und -Händler (UK)

Datenqualität und Bereinigung

Das Rohdataset enthält verschiedene Anomalien, die vor der Analyse entfernt werden müssen:

- **Stornierungen:** Invoice-Nummern beginnend mit «C» (negative Quantities) — werden vollständig entfernt
- **Fehlende Customer IDs:** ~25% der Zeilen enthalten keine Kundennummer — für RFM nicht verwendbar, werden gefiltert
- **Negative/Null-Preise:** Testbestellungen und interne Buchungen mit $\text{Price} \leq 0$ — werden entfernt
- **Customer-ID-Normalisierung:** Kundennummern werden als Text gespeichert, damit Excel-Importwerte wie 17850.0 stabil als 17850 verarbeitet werden
- **Produktfilter:** Nicht-produktbezogene Positionen wie Versand, Bankgebühren und manuelle Korrekturen werden aus Produkt-Rankings und Sortimentsregeln ausgeschlossen

Nach der Bereinigung bleiben ca. 750'000–800'000 verwertbare Transaktionszeilen für die Analyse.

FEATURE ENGINEERING

Nach der Bereinigung wird eine `revenue`-Spalte berechnet: $\text{revenue} = \text{quantity} \times \text{price}$. Diese Spalte ist die Grundlage für alle monetären Analysen (KPIs, RFM Monetary, Produktumsatz, Forecast).

4. Technologie-Entscheide

Für dieses Projekt wurden zwei bewusste Abweichungen vom Kurs-Standard vorgenommen, die hier dokumentiert und begründet werden.

Python + Streamlit statt R + Shiny

ENTSCHEID

Der Kurs hat primär R und Shiny für interaktive Dashboards eingeführt. Für dieses Projekt wurde bewusst Python + Streamlit gewählt.

Kriterium	Python + Streamlit	R + Shiny
KI-Integration	✓ Prophet, scikit-learn, pandas sind Python-nativ	⚠ Nur via <code>reticulate</code> — fehleranfällig
Pipeline-Konsistenz	✓ Eine Sprache für alles (ETL, ML, Dashboard)	✗ Hybrid nötig: R für UI, Python für ML
Kurskonsistenz	✓ Week 08: Python (<code>SimpleAutoencoder.py</code> , Prophet-Guide)	✓ Primärsprache des Kurses
Lernkurve	✓ Geringerer Boilerplate, intuitive Reaktivität	⚠ Reactive-Programmiermodell komplex
Portfolio-Relevanz	✓ Industrie-Standard für Data Science Dashboards	⚠ Hauptsächlich akademisch/statistisch

R + Shiny wäre die richtige Wahl gewesen, wenn die KI-Komponente entfallen würde und das Projekt rein deskriptiv-statistischer Natur wäre. Da die Prophet-Zeitreihe und die RFM-Berechnung den Kern des Projekts bilden, ist Python die einheitlichere Wahl.

SQLite statt MySQL

ENTSCHEID

Der Kurs verwendete MySQL (inkl. `MysqlConnector.py`). Für dieses Projekt wurde SQLite gewählt.

Kriterium	SQLite	MySQL
Portabilität	✅ Eine einzige <code>.db</code> -Datei, kein Server nötig	❌ Server-Installation + Konfiguration
Reproduzierbarkeit	✅ Automatisch beim ersten Start generiert	❌ Manueller Import, Credentials nötig
Scope-Angemessenheit	✅ Read-only BI-Dashboard, kein Concurrent Write	⚠️ Mehrwert ohne Concurrent Access nicht vorhanden
SQL-Kompatibilität	✅ Standard SQL, identisches Abfragemodell	✅ Vollständiges RDBMS

Das Prinzip (SQL-Queries, relationales Modell, Datenbankabstraktionsschicht) ist in beiden Systemen identisch — nur die Engine ist bei SQLite leichtgewichtiger. Für ein lokales BI-Dashboard ohne gleichzeitige Schreibzugriffe mehrerer Nutzer ist SQLite die angemessene Wahl.

Verwendeter Tech Stack

Schicht	Technologie	Zweck
Rohdaten	Excel (<code>online_retail_II.xlsx</code>)	Quelldaten (UCI ML Repository)
Datenbank	SQLite (<code>retail.db</code>)	Persistenz, SQL-Abfragen
Datenverarbeitung	Python + Pandas	ETL, Bereinigung, Feature Engineering
Visualisierung	Plotly	Interaktive Charts
Dashboard	Streamlit	Web-UI, 5-Tab-Struktur
Forecasting	Prophet (Meta)	Bayesianische Zeitreihenprognose
Testing	pytest	Automatisierte Unit-Tests
Versionierung	GitHub	Source Control, Portfolio

5. Systemarchitektur

Excel (online_retail_II.xlsx) | | einmaliger Import beim ersten Start ▼ SQLite Datenbank (retail.db) | Tabelle: transactions | Spalten: invoice, stock_code, description, | quantity, invoice_date, price, | customer_id, country, revenue | | SQL-Queries | ▼ ▼ ▼ Prophet RFM Produkt- Forecasting Segmentierung Analyse (forecasting.py) (rfm_analysis.py) (product_analysis.py) | | | | ▼ KI-Entscheidungsagent (decision_agent.py) | ▼ Streamlit Dashboard (app.py) | Tab 1: Übersicht (KPIs) | | Tab 2: Forecast | | Tab 3: Kunden RFM | | Tab 4: Produkte | | Tab 5: KI-Entscheid |

Abbildung 1: Systemarchitektur des BI Entscheidungsagenten

Modulstruktur

Datei	Verantwortlichkeit
<code>src/data_processing.py</code>	Excel-Import, Bereinigung, SQLite-Befüllung. Wird einmalig ausgeführt, wenn <code>retail.db</code> noch nicht existiert.
<code>src/rfm_analysis.py</code>	RFM-Berechnung pro Kunde (Recency / Frequency / Monetary), Quintil-Scoring, Segment-Zuweisung.
<code>src/forecasting.py</code>	Monatsaggregation, Prophet-Fit, 3-Monats-Prognose mit Konfidenzintervall.
<code>src/product_analysis.py</code>	Top-10-Produkte nach Umsatz, Erkennung von Produkten mit ≥ 3 aufeinanderfolgenden Monatsrückgängen.
<code>src/customer_analysis.py</code>	Eindeutige Zuordnung eines Hauptlands pro Kunde und länderbasierte Segmentauswertung ohne Doppelzählung.
<code>src/decision_agent.py</code>	Regelbasierter Agent: kombiniert alle drei Analysen und erzeugt priorisierte Empfehlungen.
<code>app.py</code>	Streamlit-Hauptapplikation mit 5 Tabs und <code>@st.cache_data</code> - Caching aller Datenladeoperationen.

Datenbankschema

Die Datenbank enthält eine einzige Tabelle `transactions`, die alle bereinigten Transaktionen speichert:

```
CREATE TABLE transactions (  
    invoice      TEXT,  
    stock_code   TEXT,  
    description  TEXT,  
    quantity     REAL,  
    invoice_date TEXT,  -- ISO-Format: YYYY-MM-DD HH:MM:SS  
    price        REAL,  
    customer_id  TEXT,  
    country      TEXT,  
    revenue      REAL   -- Computed: quantity × price  
);
```

Das Single-Table-Design ist für dieses Projekt bewusst gewählt: Die Quelldaten sind denormalisiert (jede Zeile enthält alle Informationen), und für ein Read-only BI-Dashboard sind keine Schreiboperationen oder normalisierten Relationen nötig. Sämtliche Aggregationen werden durch SQL-Queries und Pandas direkt auf dieser Tabelle durchgeführt.

6. Datenverarbeitung und Bereinigung

Das Modul `src/data_processing.py` implementiert die vollständige ETL-Pipeline (Extract, Transform, Load) vom Excel-Rohdatensatz zur SQLite-Datenbank.

ETL-Pipeline

Extract

Beide Excel-Sheets («Year 2009-2010» und «Year 2010-2011») werden mittels `pandas.read_excel()` mit dem Parameter `sheet_name=None` eingelesen und zu einem DataFrame konkateniert. Da der Excel-Import bei 1 Million Zeilen mehrere Minuten dauern kann, wird die Datenbank nur beim ersten Start generiert — bei allen folgenden Starts wird die vorhandene `retail.db` direkt verwendet.

Transform — Bereinigungsschritte

```
def clean_dataframe(df):  
    df = df.rename(columns=COLUMN_MAP)           # Spaltennamen normalisieren  
    df = df[df['quantity'] > 0]                  # Stornierungen entfernen  
    df = df[df['price'] > 0]                     # Null/Negativpreise entfernen  
    df = df.dropna(subset=['customer_id'])       # Zeilen ohne Kundennummer  
    df['revenue'] = df['quantity'] * df['price'] # Revenue berechnen  
    return df
```

Load

Der bereinigte DataFrame wird über `pandas.DataFrame.to_sql()` in die SQLite-Datenbank geschrieben. Bei einem Re-Run wird die bestehende Tabelle ersetzt (`if_exists='replace'`).

Einstiegspunkt

Die Funktion `build_database()` wird beim Start der Streamlit-App automatisch aufgerufen:

```
def build_database(excel_path=EXCEL_PATH, db_path=DB_PATH):  
    if db_path.exists():  
        return # DB bereits vorhanden – kein Re-Import  
    df = pd.concat([pd.read_excel(excel_path, sheet_name=s)  
                    for s in ['Year 2009–2010', 'Year 2010–2011']])  
    clean_df = clean_dataframe(df)  
    load_to_sqlite(clean_df, db_path)
```

PERFORMANCE-HINWEIS

Der erste Start dauert 2–5 Minuten (abhängig von Hardware), da ~1 Million Excel-Zeilen verarbeitet werden. Streamlit zeigt während des Imports einen Spinner. Alle nachfolgenden Starts laden direkt aus der SQLite-Datenbank und sind in Sekunden bereit.

7. Analyse-Module

6.1 Prophet-Zeitreihenprognose

Das Modul `src/forecasting.py` erstellt eine 3-Monats-Umsatzprognose mithilfe der Prophet-Bibliothek von Meta (ehemals Facebook). Prophet ist ein additives Zeitreihenmodell, das Trend und jährliche Saisonalität getrennt modelliert und bei fehlenden Werten robust ist.

Datenvorbereitung

Die Transaktionsdaten werden auf Monatebene aggregiert. Prophet erwartet zwei Spalten: `ds` (Datum) und `y` (Wert). Eine Besonderheit des Datensatzes ist das Enddatum: Die Daten enden am 9. Dezember 2011 — der Dezember ist damit mit nur 9 von 31 Tagen erfasst. Würde dieser unvollständige Monat in das Training einfließen, würde Prophet einen künstlichen Umsatzeinbruch von rund 60% gegenüber dem tatsächlichen Dezemberwert lernen und entsprechend pessimistische Prognosen für die Folgemonate erzeugen.

Um diesen Effekt zu verhindern, wird der letzte Monat der Trainingsserie automatisch entfernt, falls er nicht vollständig ist:

```
# Unvollständigen letzten Monat aus dem Training ausschliessen
last_month_end = monthly['ds'].iloc[-1] + pd.offsets.MonthEnd(1)
if pd.Timestamp(end_date) < last_month_end:
    monthly = monthly.iloc[:-1]
```

Das Training erfolgt auf vollständigen Monaten (Dez 2009 – Nov 2011), die Prognose deckt die drei darauffolgenden Monate ab.

Modellierung und Prognose

```
from prophet import Prophet

model = Prophet(yearly_seasonality=2, weekly_seasonality=False, daily_seasonality=False)
model.fit(series)

future = model.make_future_dataframe(periods=3, freq='MS')
forecast = model.predict(future)
```

Der Parameter `yearly_seasonality=2` entspricht einem Fourier-Order von 2 (4 Parameter für die Saisonkomponente). Gegenüber dem Default-Wert von 10 reduziert dies das Overfitting-Risiko bei einer kurzen Trainingsserie von rund 24 Monaten erheblich.

Das Modell gibt für jeden Monat drei Werte zurück: `yhat` (Punktprognose), `yhat_lower` und `yhat_upper` (80%-Konfidenzintervall). Im Dashboard werden historische Werte als durchgezogene Linie und die Prognosewerte als gestrichelte Linie mit schraffiertem Konfidenzband dargestellt.

Bekannte Modellgrenzen

Saisonaler Rückgang im Sommer: Der Datensatz stammt von einem britischen Geschenkartikel-Versandhandel, dessen Hauptsaison im November/Dezember liegt (Weihnachtsgeschäft). Prophet erkennt dieses Muster automatisch und prognostiziert für Sommermonate (Juli–September) entsprechend niedrigere Werte. Dies ist kein Modellfehler, sondern korrektes Verhalten auf Basis der historischen Datenlage.

Negative Forecast-Werte: Prophets additives Modell berechnet den Forecast als Summe aus Trend- und Saisonalitätskomponente: `yhat = trend + yearly_seasonality`. Wenn der saisonale Korrekturfaktor in einem schwachen Monat betragsmässig grösser ist als der Trendwert, kann `yhat` rechnerisch negativ werden — ein bekanntes Artefakt additiver Zeitreihenmodelle. Da ein negativer Umsatz ökonomisch nicht sinnvoll ist, werden `yhat` und `yhat_lower` vor der Ausgabe auf 0 beschränkt:

```
forecast['yhat'] = forecast['yhat'].clip(lower=0)
forecast['yhat_lower'] = forecast['yhat_lower'].clip(lower=0)
```

6.2 RFM-Kundensegmentierung

Das Modul `src/rfm_analysis.py` segmentiert alle Kunden nach dem RFM-Modell (Recency, Frequency, Monetary). RFM ist ein etabliertes Verfahren im CRM und BI, um Kunden anhand ihres Kaufverhaltens zu klassifizieren.

RFM-Metriken

Metrik	Definition	Bedeutung
Recency (R)	Tage seit dem letzten Kauf	Niedriger Wert = aktiver Kunde
Frequency (F)	Anzahl Bestellungen	Hoher Wert = treuer Kunde
Monetary (M)	Gesamtumsatz pro Kunde	Hoher Wert = wertvoller Kunde

Quintil-Scoring

Jede Metrik wird in 5 Quintile eingeteilt (Score 1–5). Für Recency gilt: Score 5 = kürzlicher Kauf (niedrige Tage), Score 1 = lange nicht gekauft. Für Frequency und Monetary: Score 5 = höchster Wert.

Segmente

Segment	RFM-Regel	Bedeutung
Champions	$R=5, F \geq 4$	Beste Kunden: kürzlich, häufig, hoher Umsatz
Loyal	$F \geq 3, R \geq 3$	Regelmässige Käufer mit guter Recency
At Risk	$R \leq 2, F \geq 3$	Früher aktiv, jetzt abgewandert — Reaktivierung nötig
Lost	$R=1, F \leq 2$	Inaktiv, kaum Kaufhistorie — schwer zurückzugewinnen
New	$R \geq 4, F=1$	Neue Kunden — Bindung aufbauen
Others	Kein Match	Mittelsegment ohne klares Profil

6.3 Produkt-Performance

Das Modul `src/product_analysis.py` liefert zwei Analysen:

Top-10-Produkte

Aggregation des Gesamtumsatzes je Produkt (StockCode + Description), Auswahl der 10 grössten Positionen. Vor der Aggregation werden nicht-produktbezogene StockCodes wie Versand, Bankgebühren, manuelle Korrekturen und Dotcom-Porto ausgeschlossen. Dadurch zeigt das Ranking echte Sortimentsartikel und keine Nebenkostenpositionen. Dargestellt wird das Ergebnis als horizontales Balkendiagramm im Dashboard.

Rückläufige Produkte

Für jeden Monat wird der Monatsumsatz je Produkt berechnet. Anschliessend wird geprüft, ob der Umsatz über ≥ 3 aufeinanderfolgende Monate konsistent gesunken ist. Diese Produkte gelten als «rückläufig» und werden dem Entscheidungsagenten übergeben.

```
def get_declining_products(monthly_df, months=3):
    # Für jedes Produkt: prüfe ob die letzten `months` Monate
    # jeweils niedriger als der Vormonat waren
    declining = []
    for product, grp in monthly_df.groupby('description'):
        last_n = grp.sort_values('month').tail(months + 1)['revenue'].values
        if all(last_n[i] > last_n[i+1] for i in range(len(last_n)-1)):
            declining.append(product)
    return declining
```

6.4 Länderbasierte Kundenauswertung

Für den Tab «Kunden RFM» wird jedem Kunden genau ein Hauptland zugeordnet. Falls ein Kunde in mehreren Ländern vorkommt, wird das Land mit dem höchsten Umsatz im gewählten Zeitraum verwendet. Dadurch werden Kunden und Monetary-Werte in der Länderübersicht nicht mehrfach gezählt. Diese Logik ist in `src/customer_analysis.py` gekapselt und wird separat getestet.

8. KI-Entscheidungsagent

Das Herzstück des Projekts ist der regelbasierte Entscheidungsagent in `src/decision_agent.py`. Er kombiniert die Ergebnisse aller drei Analyse-Module (Forecast, RFM, Produktanalyse) und leitet daraus priorisierte Managemententscheide ab.

Design-Entscheid: Regelbasiert statt LLM

Für BI-Dashboards in der Praxis ist ein regelbasierter Agent einer LLM-Integration in mehreren Punkten überlegen:

- **Transparenz:** Jede Empfehlung ist direkt auf konkrete Metriken und Schwellwerte zurückführbar — kein «Black-Box»-Verhalten
- **Determinismus:** Gleiche Datenlage ergibt immer die gleiche Empfehlung — reproduzierbar und auditierbar
- **Keine Latenz/Kosten:** Kein API-Call nötig, Empfehlungen werden in Millisekunden generiert
- **Kurskonformität:** Der Fokus liegt auf der BI-Methodik, nicht auf dem Prompt-Engineering

Entscheidungsregeln

REGEL 1 — REAKTIVIERUNGSKAMPAGNE **HOCH**

Bedingung: Forecast nächster Monat < letzter Istwert um >5% **UND** At-Risk-Kunden-Anteil >20%

Entscheid: Sofortige Reaktivierungskampagne für At-Risk-Segment lancieren

Logik: Rückläufiger Umsatz kombiniert mit einer grossen Gruppe abgewandelter Kunden signalisiert ein akutes Churn-Problem, das durch gezielte Retention-Massnahmen (z.B. Rabattcodes, persönliche E-Mails) adressiert werden kann.

REGEL 2 — SORTIMENTSBEREINIGUNG MITTEL

Bedingung: Mindestens 1 Produkt mit ≥ 3 aufeinanderfolgenden Monatsrückgängen und letztem Monatsumsatz unter 50% des Produktdurchschnitts

Entscheid: Rückläufige Produkte aus dem Sortiment nehmen oder repositionieren

Logik: Ein anhaltender Umsatzrückgang über 3+ Monate ist kein saisonales Muster, sondern ein strukturelles Problem (sinkende Nachfrage, Qualitätsprobleme, Wettbewerb).

REGEL 3 — KUNDENBINDUNG STÄRKEN MITTEL

Bedingung: Champion-Anteil $< 10\%$ der Kundenbasis

Entscheid: Kundenbindungsprogramm aufbauen und Loyal-Kunden aktiv zu Champions entwickeln

Logik: Ein zu kleiner Champion-Anteil schwächt Wiederholungsumsätze und erhöht die Abhängigkeit von kurzfristiger Akquisition.

REGEL 4 — NEUKUNDENAKQUISITION PRÜFEN MITTEL

Bedingung: Neukunden-Anteil $< 5\%$ der Kundenbasis

Entscheid: Marketing-Kanäle und Erstbestellangebote prüfen und ausbauen

Logik: Ohne kontinuierlichen Neukunden-Zufluss schrumpft der Kundenstamm langfristig durch natürliche Abwanderung.

REGEL 5 — KLUMPENRISIKO REDUZIEREN MITTEL

Bedingung: Top-20%-Kunden generieren $> 80\%$ des Umsatzes

Entscheid: Kundenstamm diversifizieren und Abhängigkeit von wenigen Schlüsselkunden reduzieren

Logik: Hohe Umsatzkonzentration macht das Geschäft anfällig für den Verlust einzelner Grosskunden.

REGEL 6 — KEIN HANDLUNGSBEDARF **TIEF**

Bedingung: Keine der Risiko- oder Optimierungsregeln greift

Entscheid: Aktuelle Strategie beibehalten und regulär überwachen

Logik: Stabile Forecasts, gesunde Kundensegmente, keine signifikant rückläufigen Produkte und geringe Umsatzkonzentration sprechen gegen kurzfristigen Handlungsbedarf.

Empfehlungsformat

Jede Empfehlung wird als strukturiertes Dictionary zurückgegeben:

```
{
  "priority":  "HOCH",          # HOCH / MITTEL / TIEF
  "finding":   "Forecast -8% ...", # Was die Daten zeigen
  "decision":  "Reaktivierungskampagne ...", # Konkrete Massnahme
  "reasoning": "Forecast: -8%, At-Risk: 23% ..." # Welche Metriken
}
```

Mehrere Regeln können gleichzeitig zutreffen. In diesem Fall werden alle aktiven Empfehlungen zurückgegeben. Die Regel mit Priorität HOCH erscheint zuerst, weil sie vor den mittleren Optimierungsregeln geprüft wird.

Schwellwerte und Kalibrierung

Die Regeln verwenden bewusst explizite Management-Schwellwerte statt eines Black-Box-Modells. Die Standardwerte sind im Code als `AgentThresholds` dokumentiert: -5% Forecast-Rückgang, 20% At-Risk-Anteil, 50% Produktumsatz gegenüber Durchschnitt, 10% Champion-Anteil, 5% Neukunden-Anteil und 80% Umsatzanteil der Top-20%-Kunden. Im Dashboard können die beiden Schwellwerte der Hoch-Prioritätsregel interaktiv verändert werden, um Sensitivität und Konsequenz der Entscheidung transparent zu machen.

9. Streamlit Dashboard

Das Frontend ist als Streamlit-Applikation mit 5 Tabs implementiert. Alle Dateladeoperationen sind mit `@st.cache_data` dekoriert, sodass die SQLite-Queries und Prophet-Berechnungen nur einmal pro Session ausgeführt werden.

Tab-Struktur

Tab	Inhalt	Zielgruppe
1 — Übersicht	4 KPI-Kacheln (Umsatz, aktive Kunden, At-Risk-Kunden, Forecast nächster Monat), Mini-Trendchart, Kundensegment-Balken, kompakte KI-Alert-Box	Management, Executive View
2 — Forecast	Historischer Monatsumsatz als Linie, Prophet-Prognose mit Konfidenzband, 3 Forecast-Metriken als Kacheln	Vertrieb, Controlling
3 — Kunden RFM	Scatter Plot (Recency vs. Frequency, Grösse = Monetary), Segmentzusammenfassung als Tabelle, At-Risk-Kundenliste, Kundensegmente nach Land	Marketing, CRM
4 — Produkte	Top-10 horizontales Balkendiagramm, Tabelle rückläufiger Produkte, Produkt-Drilldown, Umsatz nach Land	Category Management, Einkauf
5 — KI-Entscheid	Farbkodierte Empfehlungskarten (Rot=HOCH, Gelb=MITTEL, Grün=TIEF), vollständige Begründung, Live-Regelstatus, anpassbare Schwellwerte für Regel 1	Management, Entscheidungsträger

Technische Implementierung

Caching

```
@st.cache_data
def load_all(start_date, end_date, countries):
    conn = get_connection()
    rfm = load_rfm(conn, start_date, end_date, countries)
    actuals, forecast = load_forecast(conn, start_date, end_date, countries)
    top_prod, declining = load_product_analysis(conn, start_date, end_date, countries)
    recs = generate_recommendations(forecast, rfm, declining, actuals_df=actuals)
    return rfm, actuals, forecast, top_prod, declining, recs
```

Prioritäts-Farbkodierung (Tab 5)

```
PRIORITY_COLORS = {
    'HOCH': ('#fee2e2', '#991b1b'), # Rot
    'MITTEL': ('#fef9c3', '#854d0e'), # Gelb
    'TIEF': ('#dcfce7', '#166534'), # Grün
}

for rec in recommendations:
    bg, fg = PRIORITY_COLORS[rec['priority']]
    st.markdown(f"""
    <div style="background:{bg}; border-left: 4px solid {fg}; padding:16px;">
        <strong style="color:{fg}">{rec['priority']}: {rec['decision']}</strong>
        <p>{rec['reasoning']}</p>
    </div>""", unsafe_allow_html=True)
```

10. Testing und Qualitätssicherung

Das Projekt verwendet pytest für automatisierte Qualitätssicherung. Alle Analyse- und Agentenmodule haben dedizierte Testdateien. Insgesamt sind 42 Tests implementiert, die

vollständig auf synthetischen Daten laufen — keine echte Datenbank oder Excel-Datei nötig.

Testübersicht

Testdatei	Anzahl Tests	Abgedeckte Fälle
tests/test_data_processing.py	6	Bereinigung, Stornierungen, fehlende IDs, Revenue-Berechnung, DB-Schreiben/Lesen
tests/test_rfm_analysis.py	7	RFM-Berechnung, Segmentzuweisung (Champions, At Risk, Lost, New, Loyal, Others), Scoring bei kleinen Datenmengen
tests/test_forecasting.py	5	Prophet-Forecast Ausgabestruktur, Prognosehorizont, Backtest-Metriken, Grenzfall mit zu wenigen Daten
tests/test_product_analysis.py	5	Top-N-Selektion, rückläufige Produkte (3-Monats-Regel), Sonderpositionsfilter, Grenzfälle
tests/test_customer_analysis.py	2	Hauptland pro Kunde, länderbasierte Segmentauswertung ohne Doppelzählung
tests/test_decision_agent.py	17	Regeln 1–6, Schwellwerte, dünne Datenbasis, negativer Forecast, erster zukünftiger Forecast-Monat, Empfehlungsformat
Total	42	

Testausführung

```
$ pytest tests/ -v
===== test session starts =====
tests/test_customer_analysis.py .. [ 2/42]
tests/test_data_processing.py ..... [ 8/42]
tests/test_decision_agent.py ..... [ 25/42]
tests/test_forecasting.py ..... [ 30/42]
tests/test_product_analysis.py ..... [ 35/42]
tests/test_rfm_analysis.py ..... [ 42/42]
===== 42 passed in 10.09s =====
```

DESIGN-ENTSCHEID: IN-MEMORY TESTS

Alle Tests verwenden synthetische DataFrames und SQLite-In-Memory-Datenbanken (`sqlite3.connect(':memory:')`). Dies macht die Tests unabhängig vom echten Dataset (das aus Grösse-Gründen nicht ins Repo eingecheckt wird) und ermöglicht schnelle, isolierte Ausführung.

11. Fazit und Ausblick

Ergebnis

Das Projekt hat alle Anforderungen vollständig umgesetzt: Die Datenpipeline überführt 1,07 Millionen Rohtransaktionen automatisch in eine SQLite-Datenbank; drei Analyse-Module (Prophet-Forecast, RFM-Segmentierung, Produktanalyse) liefern betriebswirtschaftlich relevante Erkenntnisse; der regelbasierte Entscheidungsagent fasst diese zu priorisierten Managemententscheiden zusammen; das 5-Tab-Streamlit-Dashboard visualisiert alle Resultate interaktiv.

Das Projekt ist vollständig reproduzierbar: Nach dem Klonen des Repositorys und dem Download des Datasets genügt ein einziger Befehl (`streamlit run app.py`), um das Dashboard zu starten — ohne Server, ohne manuelle Datenbankkonfiguration, ohne weitere Setup-Schritte.

Erkenntnisse

- **Datenqualität ist entscheidend:** ~25–30% der Rohtransaktionen werden in der Bereinigung entfernt (Stornierungen, fehlende Kundennummern, Null- und Negativpreise). Ohne diese Bereinigung würden alle Analysen verzerrte Resultate liefern.
- **BI-Output braucht fachliche Filter:** Versand, Bankgebühren und manuelle Korrekturen dürfen nicht als Top-Produkte oder Sortimentsrisiken interpretiert werden.
- **Länderanalysen brauchen eindeutige Kundenzuordnung:** Ein Hauptland pro Kunde verhindert Doppelzählung bei international aktiven Kunden.
- **Prophet ist saisonal robust:** Das Modell erkennt die Weihnachts-Saisonalität des Geschenkartikel-Shops automatisch und berücksichtigt sie in der Prognose.
- **RFM ist simpel aber wirksam:** Mit nur drei Metriken und einem Quintil-Scoring lassen sich Kunden sinnvoll für gezielte Massnahmen segmentieren.
- **Regelbasierter Agent genügt für BI:** Für strukturierte Management-Entscheide auf der Grundlage klarer Metriken ist ein LLM nicht nötig — Transparenz und Determinismus sind wichtiger als Flexibilität.

Limitationen

- **Historischer Datensatz:** Die Daten enden am 09.12.2011. Forecasts zeigen daher Modellverhalten auf historischen Retail-Daten, keine aktuelle Marktentwicklung.
- **Keine Kausalität:** Die Empfehlungen identifizieren auffällige Muster, beweisen aber keine Ursachen für Umsatzrückgänge oder Kundenabwanderung.
- **Regelbasierte Heuristik:** Die Schwellwerte sind transparent und anpassbar, aber nicht automatisch statistisch optimiert.
- **Keine externen Einflussfaktoren:** Marketingaktionen, Lagerbestand, Wettbewerb und Makroeffekte sind im Datensatz nicht enthalten.

Mögliche Erweiterungen

- **E-Mail-Alert:** Automatischer Versand der KI-Empfehlung bei Überschreiten der Schwellwerte
- **Erweiterter Agent:** Zusätzliche Regeln, z.B. saisonale Produktvorschläge oder länderspezifische Massnahmen auf Basis der bestehenden Country-Auswertungen
- **Export-Funktion:** Download der aktuellen Empfehlung und gefilterten Tabellen als PDF oder Excel
- **Cloud-Deployment:** Deployment auf Streamlit Community Cloud für öffentlichen Zugang

REPOSITORY

Der vollständige Quellcode, die Testdateien, dieses Dokument und eine detaillierte Setup-Anleitung sind auf GitHub verfügbar:

<https://github.com/SheylaSam/entscheidungsagent-bi>

Sheyla Sampietro — Business Intelligence Projekt, HS 2026 — Dozent: Manuel Renold

Dataset: Chen, D. (2015). Online Retail II. UCI Machine Learning Repository. <https://doi.org/10.24432/C5CG6D>